AI/ML Engineer — Technical Assessment

Multimodal Retrieval (Qwen-VL + Streamlit) · MCP Database Agent · Reflection

Author: **Keerthivanan** · keerthivanan.ds.ai@gmail.com

Executive Summary

This submission implements the two required systems and a written reflection. **Task 1** is a multimodal retrieval system that makes PDFs *and* images searchable by meaning: a Qwen-VL vision model reads charts, diagrams and screenshots into text, which is embedded and indexed for semantic search, then compared head-to-head against a text-only RAG baseline with a quantitative evaluation. **Task 2** exposes a database to an LLM through the Model Context Protocol (MCP) and drives a plan■act■observe agent with read-only guardrails and error recovery. **Task 3** is the reflection below.

Headline Task 1 result (measured, live models): on the labelled query set, the multimodal pipeline reached **Recall@1 = 100%** overall and **100%** on the visual-answer subset, versus **50%** and **18%** for the text-only baseline — because the baseline is structurally blind to image-only documents.

Task 1 — Multimodal Retrieval with Qwen-VL + Streamlit

Objective

Build a text + image retrieval prototype using a Qwen-VL family model, exposed through Streamlit, over intranet-style documents, and evaluate it against a text-only RAG baseline.

Architecture (LangChain)

- **Ingestion** — PyMuPDF extracts selectable text from PDFs; every page image and standalone image is captioned by **Qwen2.5-VL**, turning pixels into searchable text.
- **Indexing** — text chunks and image captions are embedded and stored in a **FAISS** vector index (exact cosine).
- **Retrieval** — one shared Pipeline class builds two indexes: **multimodal** (text + captions) and **text-only baseline** (text chunks only), so the comparison is fair by construction.
- **Queries** — text queries embed directly; image queries are captioned by Qwen-VL, then embedded and searched. An optional Qwen-VL rerank scores candidate images against the query.
- **UI** — Streamlit: upload/select docs, text or image query, top-K result cards with a relevance meter and the matched page/region.

Technology & the judgment calls behind it

Decision	Choice	Why / trade-off
Vision model	Qwen2.5-VL (Ollama local; hosted API optional)	Mandated family; runs free locally on a 4GB GPU. Cloud uses a hosted Qwen
Local vs API	Local-first, pluggable	Free, private, reproducible; a reasoned API choice is a one-line swap.
Embeddings	OpenAI text-embedding-3-small; local HuggingFace fallback	Top quality with a free, strong open-source model with none. Both L2-normali
Embedding strategy	Parsed-text + image-caption	Unifies text and pixels in one vector space; Qwen-VL is load-bearing. Gives u
Vector store	FAISS (flat/exact), InMemory fallback	Real vector index; flat because at this corpus size ANN buys nothing. HNSW/

Corpus (22 documents — engineered + real)

12 engineered intranet documents (policy PDFs, charts, an org chart, a scanned salary table, an ops dashboard, an error screenshot) plus **10 real open documents** downloaded from arXiv (Transformer, ResNet, BERT, ViT, GPT-3, word2vec) and Wikimedia Commons (internet map, sunspot chart, DNA diagram, Moore's-law chart). The engineered set deliberately plants answers that exist *only* in pixels, so multimodal can measurably win.

Evaluation (18 labelled queries, 11 image-answer)

Metric	Multimodal	Text-only baseline
Recall@1 (overall)	100%	50%
Recall@1 (visual subset)	100%	18%
MRR (visual subset)	1.00	0.18
Latency p50	~335 ms	~360 ms

Failure modes documented: VLM caption hallucination (mitigated with low temperature + optional rerank); high-res images overflowing the vision context window (fixed by downscaling + larger context); embedding confusability on near-duplicate topics (resolved by the Qwen-VL rerank). Latency: image captioning is the cost (~5–15 s/image); text queries are ~50 ms.

How to run

- **Local:** `pip install -r requirements.txt` → `python make_corpus.py` → `python rag.py` (ingest) → `python evaluate.py` → `streamlit run app.py`.
- **Cloud (Render):** New → Blueprint → repo (reads render.yaml) → set OPENAI_API_KEY → deploy. The committed index + page thumbnails let it run without a GPU.
- **Graceful fallback:** with no keys/Ollama it runs in mock mode, so it never crashes — a reviewer can reproduce it in minutes.

Task 2 — MCP Database Connector + Agentic Retrieval

A dummy SQLite database (related employees / projects / issues tables with foreign keys and realistic data) is exposed to the LLM through the **Model Context Protocol (MCP)** as tools — `list_tables`, `describe_schema`, `run_query`. A small agent runs a **plan** → **act** → **observe** loop: it discovers the schema at runtime, plans a query, executes it through MCP, and returns a grounded natural-language answer — handling multi-table JOIN questions and a follow-up/clarifying step.

Why MCP (the connector layer)

MCP sits between the LLM and the database as an abstraction and security boundary. The agent never receives connection strings or executes arbitrary SQL directly — credentials live only in the MCP server's environment, and the server enforces **read-only access** (rejecting INSERT/UPDATE/DELETE/DROP) plus a row/cost limit. This is defence in depth: prompt-level intent, MCP tool-level validation, and DB-level permissions. Compared with handing the model raw credentials, MCP gives a typed, auditable, least-privilege surface with error recovery (a failed query returns an error the agent reads and retries).

Note: Task 2 was implemented separately; this section summarizes its design at the level the brief requires — refer to the Task 2 repository/notebook for the exact code and demo.

Task 3 — Reflection

1 One more week — the single highest-impact change across both tasks

A shared, automated **evaluation + observability layer** that turns “it works on my examples” into a measured, regression-tested quality bar. For Task 1: expand the labelled set and A/B a **hybrid retriever** (CLIP page-image embeddings alongside Qwen-VL captions) to measure what caption-only gives up on pure visual similarity. For Task 2: a labelled NL→result set scoring answer accuracy and error-recovery rate. Both tasks currently prove correctness on a few hand-picked cases; making quality measurable is the highest-leverage step — every later improvement then becomes data-driven, not vibes.

2 One thing I am not happy with — and the production fix

Task 1's multimodal index trusts VLM captions with no verification, and is rebuilt in memory each start. A hallucinated caption silently poisons the index, and a page cap leaves deep figures unindexed. Production approach: **(1)** verify/ground captions against OCR or a second pass and confidence-gate weak ones; **(2)** persist embeddings in an incremental vector DB (pgvector) instead of recomputing on boot; **(3)** caption all pages via an async, budgeted queue with figure-detection; **(4)** add click-through feedback to tune ranking. A defensible prototype trade-off (speed/cost over completeness) — but not something I would ship as-is.

3 When is multimodal retrieval worth it over text-only?

It comes down to one number: the fraction of answer-bearing content that lives in **non-extractable visual form**. **Worth it** when charts, diagrams, scanned tables, screenshots or dashboards carry the answers (financial/research reports, ops, healthcare, manufacturing) — my measured visual-subset Recall@1 of 100% vs 18% quantifies the payoff. **Skip it** when the corpus is predominantly born-digital text (wikis, tickets, code, emails, selectable-text PDFs): text-only RAG matches multimodal at a fraction of the cost/latency, and cheap OCR handles the little visual text. The rule I'd give a team: measure the visual-only query fraction first; if it's low, start text-only and add multimodal selectively.

Links

- **Task 1 — Multimodal Retrieval:** github.com/keerthivanan/Multimodal-Retrieval-with-Qwen-VL
- **Task 2 — MCP Database Agent:** github.com/keerthivanan/MCP-Database-connector-Agentic-retrievel
- **Task 3 — Reflection:** section above, and as REFLECTION.md in the Task 1 repository.
- **Task 1 live demo:** deployed on Render from the repo blueprint (render.yaml); runs locally with a documented one-command setup.